

Inheritance:

- Inheritance can be defined as the process where one class acquires the properties of another class
- The class which inherits the properties of other is known as subclass (derived class, child class) and the class whose properties are inherited is known as superclass (base class, parent class).
- Inheritance represents the **IS-A relationship**, also known as *parent-child* relationship.
- Advantages of inheritance:
 - Code reusability
 - Used in method overriding (so runtime polymorphism can be achieved).

Syntax:

```
class Subclass-name extends Superclass-name
{
    //methods and fields
}
```

The extends keyword indicates that you are making a new class that derives from an existing class.

Example:

```
// A simple example of inheritance.
// Create a superclass.
class A
{
    int i, j;
    void showij()
    {
        System.out.println("i and j: " + i + " " + j);
    }
}
// Create a subclass by extending class A.
class B extends A
{
    int k;
    void showk()
    {
        System.out.println("k: " + k);
    }
    void sum()
    {
```

```
        System.out.println("i+j+k: " + (i+j+k));
    }
}
class SimpleInheritance
{
    public static void main(String args [])
    {
        B subOb = new B();

        /* The subclass has access to all public members of its superclass. */
        subOb.i = 7;
        subOb.j = 8;
        subOb.k = 9;

        System.out.println("Contents of subOb: ");
        subOb.showij();
        subOb.showk();
        System.out.println();
        System.out.println("Sum of i, j and k in subOb:");
        subOb.sum();
    }
}
```

```
D:\Materials\JAVA Material\Unit 2\Inheritance>java SimpleInheritance
Contents of subOb:
i and j: 7 8
k: 9

Sum of i, j and k in subOb:
i+j+k: 24
```

Member Access and Inheritance

Although a subclass includes all of the members of its super class, it cannot access those members of the super class that have been declared as **private**. For example, consider the following simple class hierarchy:

/* In a class hierarchy, private members remain private to their class. This program contains an error and will not compile. */

```
class A
{
    int i; // public by default
    private int j; //Private to A
```

```
        void setij(int x, int y)
        {
            i = x;
            j = y;
        }
    }
    class B extends A
    {
        int total;
        void sum()
        {
            total = i + j; //A's j is not accessible here
        }
    }
    class SimpleInheritance2
    {
        public static void main(String args[])
        {
            B subOb = new B();
            subOb.setij(10, 12);
            subOb.sum();
            System.out.println("Total is " + subOb.total);
        }
    }
```

```
D:\Materials\JAVA Material\Unit 2\Inheritance>javac SimpleInheritance2.java
SimpleInheritance2.java:22: error: j has private access in A
        total = i + j; //A's j is not accessible here
                      ^
1 error
```

A super class variable can reference a subclass object:

A reference variable of a super class can be assigned a reference to any subclass derived from that super class.

Ex:

```
class A
{
    void callMe()
    {
        System.out.print("Hello");
    }
}
```

```
class B extends A
{
    void callMe()
    {
        System.out.print("Hi ");
    }
}
class Reference
{
    public static void main(String args[])
    {
        A ref;
        B b = new B();
        ref = b;
        ref.callMe();
    }
}
```

Output: Hi

Using super

Whenever the derived class inherits the base class features, there is a possibility that base class features are similar to derived class features and JVM gets an ambiguity. To overcome this, super is used to refer super class properties.

The **super** keyword in java is a reference variable that is used to refer parent class. The keyword “super” came into the picture with the concept of Inheritance. It is majorly used in the following contexts:

- super is used to refer immediate parent class instance variable.
 super.parent_instance_variable_name;
- super is used to invoke immediate parent class method
 super.parent_class_method_name();
- super is used to invoke immediate parent class constructor.
 super(arglist); // parameterized constructor
 super(); //default

Usage 1. - Using super to refer super class property

```
class Vehicle
{
    int speed=50;
}
class Bike extends Vehicle
{
    int speed=100;
    void display()
    {
        System.out.println("Vehicle Speed:"+super.speed);//will print speed of vehicle
        System.out.println("Bike Speed:"+speed);//will print speed of bike
    }
}
class SuperVariable
{
    public static void main(String args[])
    {
        Bike b=new Bike();
        b.display();
    }
}
```

Output:

```
D:\Materials\JAVA Material\Unit 2\UseofSuper>java SuperVariable
Vehicle Speed:50
Bike Speed:100
```

Usage 2. - super is used to invoke immediate parent class method

```
class Student
{
    void message()
    {
        System.out.println("Good Morning Sir");
    }
}

class Faculty extends Student
{
}
```

```
        void message()
        {
            System.out.println("Good Morning Students");
        }
        void display()
        {
            message();//will invoke or call current class message() method
            super.message();//will invoke or call parent class message() method
        }
    }
}
class SuperClassMethod
{
    public static void main(String args[])
    {
        Faculty f=new Faculty();
        f.display();
    }
}
```

Output:

```
D:\Materials\JAVA Material\Unit 2\UseofSuper>java SuperClassMethod
Good Morning Students
Good Morning Sir
```

Usage 3. - super is used to invoke immediate parent class constructor

```
class Vehicle
{
    Vehicle()
    {
        System.out.println("Vehicle is created");
    }
}

class Bike extends Vehicle
{
    Bike()
    {
        super();//will invoke parent class constructor
        System.out.println("Bike is created");
    }
}

class SuperClassConst
{
}
```

```
        public static void main(String args[])
        {
            Bike b=new Bike();
        }
    }
```

Output:

```
D:\Materials\JAVA Material\Unit 2\UseofSuper>java SuperClassConst
Vehicle is created
Bike is created
```

Calling Constructors:

Constructors are called in order of derivation, from super class to sub class. Further, since **super()** must be the first statement executed in a subclass' constructor, this order is the same whether or not **super()** is used. If **super()** is not used, then the default constructor of each super class will be executed. The following program illustrates when constructors are executed:

```
// Create a super class.
class A {
    A() {
        System.out.println("Inside A's constructor.");
    }
}
// Create a subclass by extending class A.
class B extends A
{
    B() {
        System.out.println("Inside B's constructor.");
    }
}
// Create another subclass by extending B.
class C extends B {
    C() {
        System.out.println("Inside C's constructor.");
    }
}
class CallingConstructor
{
    public static void main(String args[])
    {
```

```
        C c = new C();
    }
}
```

Output:

```
D:\Materials\JAVA Material\Unit 2\UseofSuper>java CallingConstructor
Inside A's constructor.
Inside B's constructor.
Inside C's constructor.
```

Method Overriding:

In a class hierarchy, when a method in a subclass has the same name and type signature as a method in its superclass, then the method in the subclass is said to *override* the method in the superclass.

When an overridden method is called from within a subclass, it will always refer to the version of that method defined by the subclass. The version of the method defined by the superclass will be hidden.

Example:

```
// Method overriding.
class A
{
    int i, j;
    A(int a, int b)
    {
        i = a;
        j = b;
    }
    // display i and j
    void show()
    {
        System.out.println("i and j: " + i + " " + j);
    }
}

class B extends A
{
    int k;
    B(int a, int b, int c)
    {
        super(a, b);
    }
}
```



```
        k = c;
    }
    // display k – this overrides show() in A
    void show()
    {
        System.out.println("k: " + k);
    }
}
class Override
{
    public static void main(String args[]) {
        B subOb = new B(1, 2, 3);
        subOb.show(); // this calls show() in B
    }
}
```

Output:

```
D:\Materials\JAVA Material\Unit 2\UseofSuper>java Override
k: 3
```

Dynamic Method Dispatch:

Dynamic method dispatch is the mechanism by which a call to an overridden method is resolved at run time, rather than compile time. Dynamic method dispatch is important because this is how Java implements run-time polymorphism.

Example: //A Superclass Variable Can Reference to a Subclass Object

```
class A
{
    void callMe()
    {
        System.out.println("Inside A");
    }
}
class B extends A
{
    void callMe()
    {
        System.out.println("Inside B");
    }
}

class C extends B
{
}
```

```
        void callMe()
        {
            System.out.println("Inside C");
        }
    }
    class DynamicMethodDispatch
    {
        public static void main(String args[])
        {
            A a=new A();
            B b = new B();
            C c = new C();
            A ref;
            ref = a;
            ref.callMe();

            ref = b;
            ref.callMe();

            ref=c;
            ref.callMe();
        }
    }
```

Output:

```
D:\Materials\JAVA Material\Unit 2\Inheritance>java DynamicMethodDispatch
Inside A
Inside B
Inside C
```

Applying Method Overriding:

```
import java.util.*;

class Student
{
    int n;
    String name;
    void read()
    {
        Scanner s=new Scanner(System.in);
        System.out.println("Enter no and Name");
        n=s.nextInt();
        name=s.nextLine();
    }
}
```

```
        void show()
        {
            System.out.println("No:"+n);
            System.out.println("Name:"+name);
        }
    }
    class ITStudent extends Student
    {
        void read()
        {
            super.read();
        }
        void show()
        {
            super.show();
        }
    }
    class CSEStudent extends Student
    {
        void read()
        {
            super.read();
        }
        void show()
        {
            super.show();
        }
    }
    class ECESStudent extends Student
    {
        void read()
        {
            super.read();
        }
        void show()
        {
            super.show();
        }
    }
    class MainMethod
    {
        public static void main(String ar[])
        {
            ITStudent it=new ITStudent();
            CSEStudent cse=new CSEStudent();
```

```
        ECESStudent ece=new ECESStudent();

        Student sref;

        sref=it;
        sref.read();
        sref.show();

        sref=cse;
        sref.read();
        sref.show();

        sref=ece;
        sref.read();
        sref.show();
    }
}
```

Abstract Class

- An abstract class is a class that contains one or more abstract methods.
- An abstract method is a method without method body.

Syntax:

```
abstract return_type method_name(parameter_list);
```

- An abstract class can contain instance variables, constructors, concrete methods in addition to abstract methods.
- All the abstract methods of abstract class should be implemented in its sub classes.
- If any abstract method is not implemented in its subclasses, then that sub class must be declared as abstract.
- We cannot create an object to abstract class, but we can create reference of abstract class.
- Also, you cannot declare abstract constructors or abstract static methods.

Example 1: Java program to illustrate abstract class.

```
abstract class MyClass
{
    abstract void calculate(double x);
}
class Sub1 extends MyClass
{
    void calculate(double x)
```

```
        {
            System.out.println("Square :"+(x*x));
        }
    }
    class Sub2 extends MyClass
    {
        void calculate(double x)
        {
            System.out.println("Square Root :"+Math.sqrt(x));
        }
    }
    class Sub3 extends MyClass
    {
        void calculate(double x)
        {
            System.out.println("Cube :"+(x*x*x));
        }
    }
    class AC
    {
        public static void main(String arg[])
        {
            Sub1 obj1=new Sub1();
            Sub2 obj2=new Sub2();
            Sub3 obj3=new Sub3();

            obj1.calculate(20);
            obj2.calculate(20);
            obj3.calculate(20);
        }
    }
}
```

Example 2: Java program to illustrate abstract class.

```
// A Simple demonstration of abstract.
abstract class A
{
    abstract void callme();
    // concrete methods are still allowed in abstract classes
    void callmetoo() {
        System.out.println("This is a concrete method.");
    }
}
class B extends A
{
    void callme()
```

```
        {
            System.out.println("B's implementation of callme.");
        }
    }

    class AbstractDemo
    {
        public static void main(String args[])
        {
            B b = new B();
            b.callme();
            b.callmetoo();
        }
    }
```

Uses of Final:

Final can be used in three ways:

- To prevent modifications to the instance variable
- To Prevent method overriding
- To prevent inheritance

Use 2: To prevent method overriding

```
class Vehicle
{
    final void run()
    {
        System.out.println("running");
    }
}

class Bike extends Vehicle
{
    void run()
    {
        System.out.println("running safely with 100kmph");
    }
}

class FinalMethod
{
    public static void main(String args[])
    {
        Bike b= new Bike();
        b.run();
    }
}
```

```
    }  
}
```

Error Information:

```
D:\Materials\JAVA Material\Unit 2\Inheritance>javac FinalMethod.java  
FinalMethod.java:10: error: run() in Bike cannot override run() in Vehicle  
    void run()  
      ^  
    overridden method is final  
1 error
```

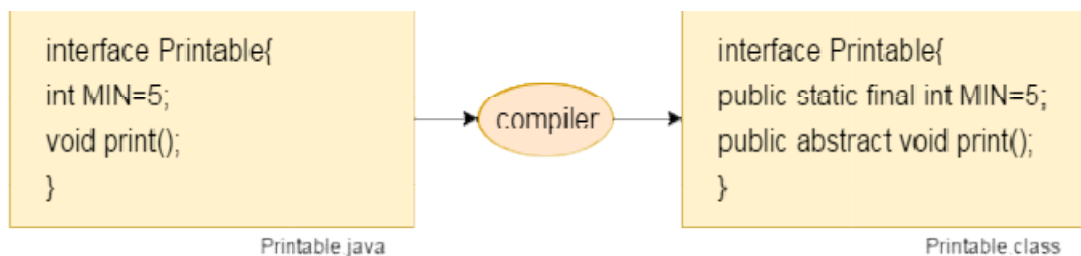
Use 3: To prevent inheritance

```
final class Vehicle  
{  
    void run()  
    {  
        System.out.println("running");  
    }  
}  
class Bike extends Vehicle  
{  
    void run()  
    {  
        System.out.println("running safely with 100kmph");  
    }  
}  
  
class FinalClass  
{  
    public static void main(String args[])  
    {  
        Bike b= new Bike();  
        b.run();  
    }  
}
```

```
D:\Materials\JAVA Material\Unit 2\Inheritance>javac FinalClass.java  
FinalClass.java:8: error: cannot inherit from final Vehicle  
class Bike extends Vehicle  
      ^  
1 error
```

Interfaces:

- A named collection of method declarations.
- A Java interface is a collection of constants and abstract methods
- Since all methods in an interface are abstract, the abstract modifier is usually left off
- Using interface, you can specify what a class must do, but not how it does.
- Interface fields are public, static and final by default, and methods are public and abstract.

**Advantages of interfaces:**

- It is used to achieve abstraction.
- By interface, we can support the functionality of multiple inheritances.

Syntax:

```
access_specifier interface interface_name
{
    return-type method-name1(parameter-list);
    return-type method-name2(parameter-list);
    type final-varname1 = value;
    type final-varname2 = value;
    //...
    return-type method-nameN(parameter-list);
    type final-varnameN = value;
}
```


Implementing Interfaces:

- Once an interface has been defined, one or more classes can implement that interface.
- To implement an interface, include the implements clause in a class definition, and then create the methods defined by the interface.
- The general form of a class that includes the **implements clause looks like this:**

```
class classname [extends superclass] [implements interface1 [,interface2...]] {  
    // class-body  
}
```
- If a class implements more than one interface, the interfaces are separated with a comma.
- The methods that implement an interface must be public. Also, the type signature of implementing method must match exactly the type signature specified in interface definition.

Example 1: Write a java program to implement interface.

```
interface Moveable  
{  
    int AVG_SPEED=30;  
    void Move();  
}  
class Move implements Moveable  
{  
    void Move(){  
        System.out.println ("Average speed is: "+AVG_SPEED );  
    }  
}  
class Vehicle  
{  
    public static void main (String[] arg)  
    {  
        Move m = new Move();  
        m.Move();  
    }  
}
```

Example 2: Write a java program to implement interface.

```
interface Teacher
{
    void display1();
}
interface Student
{
    void display2();
}
class College implements Teacher, Student
{
    public void display1()
    {
        System.out.println("Hi I am Teacher");
    }
    public void display2()
    {
        System.out.println("Hi I am Student");
    }
}
class CollegeData
{
    public static void main(String arh[])
    {
        College c=new College();
        c.display1();
        c.display2();
    }
}
```

Accessing implementations through interface references:

We can declare variables as object references that use an interface rather than a class type. Any instance of any class that implements the declared interface can be referred to by such a variable. When you call a method through one of these references, the correct version will be called based on the actual instance of the interface being referred to. This is one of the key features of interfaces. The method to be executed is looked up dynamically at run time, allowing classes to be created later than the code which calls methods on them.

Ex:

```
interface Test {  
    void call();  
}  
  
class InterfaceTest implements Test {  
    public void call()  
    {  
        System.out.println("call method called");  
    }  
}  
  
public class InterfaceReferences {  
    public static void main(String[] args)  
    {  
        Test f;  
        InterfaceTest it= new InterfaceTest();  
        f=it;  
        f.call();  
    }  
}
```

Variables in Interfaces:

We can use interfaces to import shared constants into multiple classes by simply declaring an interface that contains variables that are initialized to the desired values. When we include that interface in a class (that is, when you “implement” the interface), all of those variable names will be in scope as constants. (This is similar to using a header file in C/C++ to create a large number of **#defined** constants or **const** declarations). If an interface contains no methods, then any class that includes such an interface doesn’t actually implement anything. It is as if that class were importing the constant fields into the class name space as **final** variables.

Example: Java program to demonstrate variables in interface.

```
interface left  
{  
    int i=10;  
}  
interface right
```

```
{
    int i=100;
}
class Test implements left,right
{
    public static void main(String args[])
    {
        System.out.println(left.i);//10 will be printed
        System.out.println(right.i);//100 will be printed*/
    }
}
```

Interfaces can be extended:

One interface can inherit another by use of the keyword **extends**. The syntax is the same as for inheriting classes. When a class implements an interface that inherits another interface, it must provide implementations for all methods defined within the interface inheritance chain.

Example: Java program to demonstrate interfaces can be extended with extend keyword.

```
interface Teacher
{
    void display1();
}
interface Student
{
    void display2();
}
interface T_S extends Teacher, Student
{
    void display3();
}
class College implements T_S
{
    public void display1()
    {
        System.out.println("Hi I am Teacher");
    }
    public void display2()
    {
        System.out.println("Hi I am Student");
    }
    public void display3()
    {
        System.out.println("Hi I am Teacher_Student");
    }
}
```

```
    }  
}  
class Class_Interface  
{  
    public static void main(String arh[])  
    {  
        College c=new College();  
        c.display1();  
        c.display2();  
        c.display3();  
    }  
}
```

Example 2: Java program to implement interface and inheriting the properties from a class.

```
interface Teacher  
{  
    void display1();  
}  
class Student  
{  
    void display2()  
    {  
        System.out.println("Hi I am Student");  
    }  
}  
class College extends Student implements Teacher  
{  
    public void display1()  
    {  
        System.out.println("Hi I am Teacher");  
    }  
}  
class Interface_Class  
{  
    public static void main(String arh[])  
    {  
        College c=new College();  
        c.display1();  
        c.display2();  
    }  
}
```

Difference between Interface and Abstract class:

Abstract Class	Interface
Contains some abstract methods and some concrete methods	Only abstract methods
Contains instance variables	Only static and final variables
Doesn't support multiple inheritance	Supports
public class Apple extends Food { ... }	public class Person implements Student, Athlete, Chef { ... }

Packages:

- A Package can be defined as a grouping of related types(classes, interfaces)
- A package represents a directory that contains related group of classes and interfaces.
- Packages are used in Java in order to prevent naming conflicts.
- There are two types of packages in Java.
 1. Pre-defined Packages(built-in)
 2. User defined packages

Pre-defined Packages:

Package Name	Description
java.lang	Contains language support classes (for e.g classes which defines primitive data types, math operations, etc.). This package is automatically imported.
java.io	Contains classes for supporting input / output operations.
java.util	Contains utility classes which implement data structures like Linked List, Hash Table, Dictionary, etc and support for Date / Time operations. This package is also called as Collections .
java.applet	Contains classes for creating Applets.
java.awt	Contains classes for implementing the components of graphical user interface (like buttons, menus, etc.).

java.net	Contains classes for supporting networking operations.
javax.swing	This package helps to develop GUI like java.awt. The 'x' in javax represents that it is an extended package which means it is a package developed from another package by adding new features to it. In fact, javax.swing is an extended package of java.awt.
java.sql	This package helps to connect to databases like Oracle/Sybase/Microsoft Access to perform different operations.

Defining a Package(User defined):

To create a package is quite easy: simply include a **package** command as the first statement in a Java source file. Any classes declared within that file will belong to the specified package. The **package** statement defines a name space in which classes are stored. If you omit the **package** statement, the class names are put into the default package, which has no name.

This is the general form of the **package** statement:

```
package pkg;
```

Here, *pkg* is the name of the package.

For example, the following statement creates a package called **MyPackage**:

```
package MyPackage;
```

Java uses file system directories to store packages. For example, the **.class** files for any classes you declare to be part of **MyPackage** must be stored in a directory called **MyPackage**. Remember that case is significant, and the directory name must match the package name exactly. More than one file can include the same **package** statement.

The **package** statement simply specifies to which package the classes defined in a file belong. It does not exclude other classes in other files from being part of that same package. Most real-world packages are spread across many files. You can create a hierarchy of packages. To do so, simply separate each package name from the one above it by use of a period. The general form of a multileveled package statement is shown here:

```
package pkg1[.pkg2[.pkg3]];
```

A package hierarchy must be reflected in the file system of your Java development system. For example, a package declared as

```
package java.awt.image;
```

Example: Package demonstration

```
package pack;
public class Addition
{
    int x,y;
    public Addition(int a, int b)
    {
        x=a;
        y=b;
    }
    public void sum()
    {
        System.out.println("Sum :"+(x+y));
    }
}
```

Step 1: Save the above file with Addition.java

```
package pack;
public class Subtraction
{
    int x,y;
    public Subtraction(int a, int b)
    {
        x=a;
        y=b;
    }
    public void diff()
    {
        System.out.println("Difference :"+(x-y));
    }
}
```

Step 2: Save the above file with Subtraction.java**Step 3:** Compilation

To compile the java files use the following commands

```
javac -d directory_path name_of_the_java file
```

```
Javac -d . name_of_the_java file
```

Note: -d is a switching options creates a new directory with package name. Directory path represents in which location you want to create package and . (dot) represents current working directory.


```
D:\Materials\JAVA Material\Unit 2\PackExamples>javac -d . Addition.java
D:\Materials\JAVA Material\Unit 2\PackExamples>javac -d . Subtraction.java
```

Step 4: Access package from another package

There are three ways to use package in another package:

1. With fully qualified name.

```
class UseofPack
{
    public static void main(String arg[])
    {
        pack.Addition a=new pack.Addition(10,15);
        a.sum();
        pack.Subtraction s=new pack.Subtraction(20,15);
        s.difference();
    }
}
```

2. import package.classname;

```
import pack.Addition;
import pack.Subtraction;
class UseofPack
{
    public static void main(String arg[])
    {
        Addition a=new Addition(10,15);
        a.sum();
        Subtraction s=new Subtraction(20,15);
        s.difference();
    }
}
```

3. import package.*;

```
import pack.*;
class UseofPack
{
    public static void main(String arg[])
    {
        Addition a=new Addition(10,15);
        a.sum();
        Subtraction s=new Subtraction(20,15);
        s.difference();
    }
}
```

Note: Don't place Addition.java, Subtraction.java files parallel to the pack directory. If you place JVM searches for the class files in the current working directory not in the pack directory.

Access Protection

- Access protection defines actually how much an element (class, method, variable) is exposed to other classes and packages.
- There are four types of access specifiers available in java:
 1. Visible to the class only (private).
 2. Visible to the package (default). No modifiers are needed.
 3. Visible to the package and all subclasses (protected)
 4. Visible to the world (public)

	Private	No Modifier	Protected
Same class	Yes	Yes	Yes
Same package subclass	No	Yes	Yes
Same package non-subclass	No	Yes	Yes
Different package subclass	No	No	Yes

Example:

The following example shows all combinations of the access control modifiers. This example has two packages and five classes. The source for the first package defines three classes: **Protection**, **Derived**, and **SamePackage**.

Name of the package: pkg1

This file is Protection.java

```
package pkg1;

public class Protection
{
    int n = 1;
    private int n_priv = 2;
    protected int n_prot = 3;
    public int n_publ = 4;
```

```
        public Protection()
        {
            System.out.println("base constructor");
            System.out.println("n = " + n);
            System.out.println("n_priv = " + n_priv);
            System.out.println("n_prot = " + n_prot);
            System.out.println("n_publ = " + n_publ);
        }
    }
```

This is file Derived.java:

```
package pkg1;

class Derived extends Protection
{
    Derived()
    {
        System.out.println("Same package - derived (from base) constructor");
        System.out.println("n = " + n);

        /* class only
        * System.out.println("n_priv = " + n_priv); */

        System.out.println("n_prot = " + n_prot);
        System.out.println("n_publ = " + n_publ);
    }
}
```

This is file SamePackage.java

```
package pkg1;

class SamePackage
{
    SamePackage()
    {
        Protection pro = new Protection();
        System.out.println("same package - other constructor");
        System.out.println("n = " + pro.n);

        /* class only
        * System.out.println("n_priv = " + pro.n_priv); */

        System.out.println("n_prot = " + pro.n_prot);
    }
}
```

```
        System.out.println("n_publ = " + pro.n_publ);
    }
}
```

Name of the package: pkg2

This is file Protection2.java:

```
package pkg2;

class Protection2 extends pkg1.Protection
{
    Protection2()
    {
        System.out.println("Other package-Derived (from Package 1-Base)
        Constructor");

        /* class or package only
        * System.out.println("n = " + n); */

        /* class only
        * System.out.println("n_priv = " + n_priv); */

        System.out.println("n_prot = " + n_prot);
        System.out.println("n_publ = " + n_publ);
    }
}
```

This is file **OtherPackage.java**

```
package pkg2;

class OtherPackage
{
    OtherPackage()
    {
        pkg1.Protection pro = new pkg1.Protection();

        System.out.println("other package - Non sub class constructor");

        /* class or package only
        * System.out.println("n = " + pro.n); */

        /* class only
        * System.out.println("n_priv = " + pro.n_priv); */

        /* class, subclass or package only
```

```
        * System.out.println("n_prot = " + pro.n_prot); */  
    System.out.println("n_publ = " + pro.n_publ);  
    }  
}
```

If you want to try these two packages, here are two test files you can use. The one for package **pkg1** is shown here:

```
/* demo package pkg1 */  
  
package pkg1;  
  
/* instantiate the various classes in pkg1 */  
public class Demo  
{  
    public static void main(String args[])  
    {  
        Derived obj2 = new Derived();  
        SamePackage obj3 = new SamePackage();  
    }  
}
```

The test file for package pkg2 is

```
package pkg2;  
  
/* instantiate the various classes in pkg2 */  
public class Demo2  
{  
    public static void main(String args[])  
    {  
        Protection2 obj1 = new Protection2();  
        OtherPackage obj2 = new OtherPackage();  
    }  
}
```

```
D:\Materials\JAVA Material\Unit 2\Packages\AccessSpecifier>javac -d . Demo.java
D:\Materials\JAVA Material\Unit 2\Packages\AccessSpecifier>javac -d . Demo2.java
D:\Materials\JAVA Material\Unit 2\Packages\AccessSpecifier>java pkg1.Demo
base constructor
n = 1
n_priv = 2
n_prot = 3
n_publ = 4
Same package - derived (from base) constructor
n = 1
n_prot = 3
n_publ = 4
base constructor
n = 1
n_priv = 2
n_prot = 3
n_publ = 4
same package - other constructor
n = 1
n_prot = 3
n_publ = 4
```

```
D:\Materials\JAVA Material\Unit 2\Packages\AccessSpecifier>java pkg2.Demo2
base constructor
n = 1
n_priv = 2
n_prot = 3
n_publ = 4
Other package - Derived (from Package 1-Base)Constructor
n_prot = 3
n_publ = 4
base constructor
n = 1
n_priv = 2
n_prot = 3
n_publ = 4
other package - Non sub class constructor
n_publ = 4
```

Sealed Classes:

- Sealed class is declared using sealed keyword.
- Sealed classes allow to declare which class can be a subtype using permits keyword.
- A class extending sealed class must be declared as either sealed, non-sealed or final.
- Sealed classes helps in creating a finite and determinable hierarchy of classes in inheritance.

```
//program for sealed classes
sealed class A permits B, D {
    //body of Class A
}
sealed class B extends A permits C {
    //body of Class B
}
non - sealed class C extends B {
    //body of Class C
}
non - sealed class D extends A {
    //body of Class C
}
class SealedDemo {
    public static void main(String args[]) {
        //body of Main Function
    }
}
```

Example:

```
//program for Multilevel using sealed
sealed class A permits B {
    int a;
    A(int x) {
        a = x;
    }
    void display() {
        System.out.println("ClassA A=" + a);
    }
}
sealed class B extends A permits C {
    int b;
    B(int x, int y) {
        super(x);
        b = y;
    }
    void show() {
        System.out.println("ClassB =" + (a + b));
    }
}
```

```

    }
}
non - sealed class C extends B {
    int c;
    C(int x, int y, int z) {
        super(x, y);
        c = z;
    }
    void view() {
        System.out.println("ClassC =" + (a + b + c));
    }
}
}
class SealedClass {
    public static void main(String args[]) {
        C obj = new C(12, 16, 28);
        obj.display();
        obj.show();
        obj.view();
    }
}

```

Output:

```

ClassA A=12
ClassB =28
ClassC =56

```

Default methods in interfaces :

Before Java 8, interfaces could have only abstract methods. The implementation of these methods has to be provided in a separate class. So, if a new method is to be added in an interface, then its implementation code has to be provided in the class implementing the same interface. To overcome this issue, Java 8 has introduced the concept of default methods which allow the interfaces to have methods with implementation without affecting the classes that implement the interface

Example:

```

interface TestInterface {
    // abstract method
    public void square(int a); // default method
    default void show() {
        System.out.println("Default Method Executed");
    }
}
class TestClass implements TestInterface {
    // implementation of square abstract method
    public void square(int a) {
        System.out.println(a * a);
    }
}

```



```

    }
    public static void main(String args[]) {
        TestClass d = new TestClass();
        d.square(4);
        // default method executed d.show();
    }
}

```

Output:

16

Default Method Executed Default methods are also known as defender methods or virtual extension methods.

Static methods in interfaces:

The interfaces can have static methods as well which is similar to static method of classes.

Example:

```

interface TestInterface { // abstract method
    public void square(int a); // static method
    static void show() {
        System.out.println("Static Method Executed");
    }
}
class TestClass implements TestInterface {
    // Implementation of square abstract method
    public void square(int a) {
        System.out.println(a * a);
    }
    public static void main(String args[]) {
        TestClass d = new TestClass();
        d.square(4);
        // Static method executed
        TestInterface.show();
    }
}

```

Output:

16
Static Method Executed

Collections in Java Programming:

- The Collection in Java is a framework that provides a set of interfaces and classes to store and manipulate data efficiently.
- A Collection is defined as a single unit of objects and a framework is a ready-made architecture that provides a set of interfaces and classes.
- With the use of Collections, you can write your code quickly and efficiently. You can easily store and manipulate the data by performing operations such as searching, sorting, insertion, deletion, and traversing.
- The package `java.util.Collection` or `java.util` contains all the interfaces and their implementation known as classes and algorithms for the Collection framework. Thus, you need to import the `java.util.Collection` package to use Collection classes and interfaces in Java.

The advantages of using collections are listed below:

- By using the collections framework, you can store both homogenous and heterogeneous data.
- It also provides flexibility since its size can be expanded according to the user's requirements.
- You can perform many operations such as insertion, deletion, searching, sorting, etc. very easily by the use of methods.

▼ Collection Interface in Java

- It is at the top level of the Collection framework hierarchy.
- It is implemented by all of the collection framework's classes.
- It consists of a declaration of the methods available to all the classes that implement the collection interface.
- There are many methods of Collection interface. Listed below are some of the methods of collection interfaces method:
 - ❖ `boolean add(Object obj)`
 - ❖ `boolean addAll(Collection c)`
 - ❖ `boolean remove(Object obj)`
 - ❖ `boolean removeAll(Collection c)`
 - ❖ `int size()`
 - ❖ `void clear()`

➤ ArrayList

It is a dynamic array which means that its size can be decreased or increased as per requirement. Thus, it provides flexibility. You can randomly access the elements of the array list by using its index number.

Syntax

```
ArrayList <dataType> arraylistName=new ArrayList<>();
```

Example:

```
import java.util.*;
public class AraryListExample {
    public static void main(String []array){
        ArrayList<String> list=new ArrayList<>();

        // adding elements in an arraylist
        list.add("CSE");
        list.add("AIML");
        list.add("AIDS");
        list.add("IT");
        list.add("ECE");

        // printing elements of arraylist
        System.out.println(list);
        // accessing element of arraylist by its index number
        System.out.println(list.get(0));

        // removing element
        list.remove("AIML");
        System.out.println(list);
        System.out.println("Length of arraylist =" +list.size());

        // removing element by the index number of element.
        list.remove(0);
        System.out.println(list);

        // returns false because the element doesn't exist in arraylist.
        System.out.println(list.remove("EEE"));

        // removing all the elements of arraylist.
        list.clear();
    }
}
```

Output:

```
[CSE, AIML, AIDS, IT, ECE]
Length of arraylist=5
[CSE, AIDS, IT, ECE]
[AIDS, IT, ECE]
False
```

➤ Vector

It is similar to ArrayList, which stores elements in a dynamic array. The two key differences between ArrayList and Vector are that Vector is synchronized and has many methods that aren't included in the Collections framework.

Syntax

```
Vector <dataType> vectorName=new Vector<>();
```

Example:

```
import java.util.*;
public class VectorExample {
    public static void main(String []array){
        Vector<Float> vector=new Vector<>();

        // adding elements using add method in an arraylist
        vector.add(5.4f);
        vector.add(-4.5f);
        vector.add(9.55f);
        // adding elements using addElement method in an arraylist
        vector.addElement(6.5f);
        vector.addElement(15.76f);

        // printing vector
        System.out.println(vector);

        vector.remove(5.4f);
        System.out.println(vector);
    }
}
```

Output:

```
[5.4, -4.5, 9.55, 6.5, 15.76]
[-4.5, 9.55, 6.5, 15.76]
```

➤ Hashtable

Java Hashtable class implements a hashtable, which maps keys to values. It inherits Dictionary class and implements the Map interface.

- A Hashtable is an array of a list. Each list is known as a bucket. The position of the bucket is identified by calling the hashCode() method. A Hashtable contains values based on the key.
- Java Hashtable class contains unique elements.
- Java Hashtable class doesn't allow null key or value.
- Java Hashtable class is synchronized.

Syntax:

```
Hashtable<K,V> hashtablename=new Hashtable<K,V>();
```

K: It is the type of keys maintained by this map.

V: It is the type of mapped values.

Example:

```
import java.util.*;
public class HashDemo {
    public static void main(String[] args) {
        Hashtable<String, Integer> hashtable = new Hashtable<String, Integer>();
        // Adding elements to the hashtable
        hashtable.put("A", 1);
        hashtable.put("B", 2);
        hashtable.put("C", 3);

        // Getting values from the hashtable
        int valueA = hashtable.get("A");
        System.out.println("Value of A: " + valueA);

        // Removing elements from the hashtable
        hashtable.remove("B");

        // Enumerating the elements of the hashtable
        Enumeration<String> keys = hashtable.keys();
        while (keys.hasMoreElements()) {
            String key = keys.nextElement();
            System.out.println("Key: " + key + ", Value: " + hashtable.get(key));
        }
    }
}
```

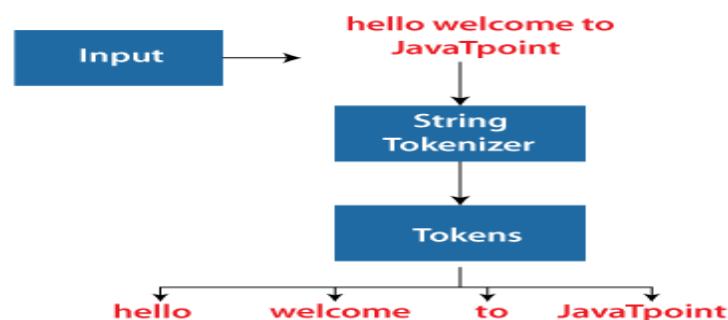
Output:

```
Value of A: 1
Key: A, Value: 1
Key: C, Value: 3
```

➤ StringTokenizer

The `java.util.StringTokenizer` class allows you to break a String into tokens. It is a simple way to break a String.

It is a legacy class of Java. In the `StringTokenizer` class, the delimiters can be provided at the time of creation or one by one to the tokens.

Example of StringTokenizer class in Java

There are 3 constructors defined in the StringTokenizer class:

StringTokenizer(String str)	It creates StringTokenizer with specified string.
StringTokenizer(String str, String delim)	It creates StringTokenizer with specified string and delimiter.
StringTokenizer(String str, String delim, boolean returnValue)	It creates StringTokenizer with specified string, delimiter and returnValue. If return value is true, delimiter characters are considered to be tokens. If it is false, delimiter characters serve to separate tokens.

The six useful methods of the StringTokenizer class are as follows:

boolean hasMoreTokens()	It checks if there is more tokens available.
String nextToken()	It returns the next token from the StringTokenizer object.
String nextToken(String delim)	It returns the next token based on the delimiter.
boolean hasMoreElements()	It is the same as hasMoreTokens() method.
Object nextElement()	It is the same as nextToken() but its return type is Object.
int countTokens()	It returns the total number of tokens.

Example:

```
import java.util.Scanner;
import java.util.StringTokenizer;

public class StringTokenizerDemo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a string to tokenize: ");
        String inputString = scanner.nextLine();
        StringTokenizer tokenizer = new StringTokenizer(inputString, " ,");
        // Tokenize using space and comma as delimiters
        int tokenCount = tokenizer.countTokens();
        if (tokenCount > 0) {
            System.out.println("Tokens found in the input string:");
            while (tokenizer.hasMoreTokens()) {
                String token = tokenizer.nextToken();
                System.out.println("Token: " + token);
            }
            System.out.println("Total number of tokens: " + tokenCount);
        } else {
            System.out.println("No tokens found in the input string.");
        }
        scanner.close();
    }
}
```

Output:

Enter a string to tokenize: I am studying 2nd year AIML in GMRIT

Tokens found in the input string:

Token: I

Token: am

Token: studying

Token: 2nd

Token: year

Token: AIML

Token: in

Token: GMRIT

Total number of tokens: 8